Benchmarking Approximate Nearest Neighbour Search

Samarth (2024101020) Assignment 3

Data	n	Single QPS	Batch QPS	Ratio	Set match
SIFT	1,000,000	30.8	342	11.1×	0.9935
GloVe	1,183,514	34.4	348	10.1×	1.0000

Table 1: Exact search, 1 thread: one query per call vs one batch call; agreement with the file's top-10 sets.

1 Setup

Machine. All numbers come from one laptop: an Intel Core i7-13700H (14 physical, 20 logical cores) with 15.75 GB of RAM, running Windows 11 (build 26200). Software: Python 3.11.17, FAISS 1.14.3 (faiss-cpu), numpy 2.4.6 and OpenBLAS 0.3.34. HNSW is FAISS's `IndexHNSWFlat`, and index builds use 20 threads.

Data. SIFT has 1,000,000 base vectors (d=128) and GloVe-100 has 1,183,514 (d=100). GloVe vectors are L2-normalised, so Euclidean order equals angular order. Each dataset has 10,000 queries: the first 5,000 are for tuning and the last 5,000 for reporting.

Recall. Recall@10 is tie-aware. A returned id is correct if its true squared distance is at most $d_{10}(1+\epsilon)+\epsilon_{\text{abs}}$, where d_{10} is the 10th exact neighbour's distance, $\epsilon = 10^{-5}$ and $\epsilon_{\text{abs}} = 10^{-6}$. Duplicate and -1 ids are ignored, and recall is $\min(\#\text{correct}, 10)/10$ averaged over queries. Distances are recomputed from the vectors in float64, never taken from the index. Ground truth is our own exact top-100. The function has 11 unit tests.

Protocol.

- QPS is measured one query per call on a single thread (one FAISS and one BLAS thread), pinned to a performance core.
- Each measurement is one warm-up pass plus 3 timed passes; we report the median.
- Tuning uses the first 2,000 tune queries. An operating point at a target recall is the fastest config whose *tune* recall reaches it; its *test* recall and QPS are reported as measured.
- A sweep stops after two consecutive points with test recall $\geq$0.999.
- Before each sweep, a fixed exact search (200 queries on 20,000 vectors) is timed as a canary for machine-state drift.

2 Part 1: Exact baseline

Agreement. Our exact top-10 sets match the file for 99.35% of SIFT queries and every GloVe query (Table 1). All 65 SIFT mismatches are ties at the 10th distance, which SIFT's integer coordinates make common. With ties allowed, recall against the file is 1.0000 on both datasets, in both directions. The file's distances match ours to a relative 1.2×10^{-7} on SIFT and 1.6×10^{-7} on GloVe.

Single query vs batch. On one thread, one batch call is 11.1× (SIFT) and 10.1× (GloVe) faster than one query per call.

- *Batch:* FAISS turns a batch into one matrix–matrix product (BLAS GEMM). Each base vector is loaded once and reused from cache for many queries, with SIMD-friendly blocking.
- *One at a time:* each query is a matrix–vector product. It re-streams every base vector (512 B on SIFT) from RAM, so it is memory-bandwidth bound, and it also pays a per-call overhead.

With 20 threads the batch reaches 761 QPS on SIFT but only 383 on GloVe; we did not find out why.

3 Part 2: HNSW, IVF-Flat and IVF-PQ

What was run.

- HNSW: $M \in \{16, 32, 48\}$, sweeping efSearch up to 512.
- IVF-Flat: nlist $\in \{1024, 4096, 8192\}$, sweeping nprobe up to 512.
- IVF-PQ: nlist $\in \{1024, 4096\}$ with 8-bit codes, $m \in \{16, 32, 64\}$ (SIFT) or $\{20, 25, 50\}$ (GloVe).
- Two GloVe sweeps that never reached 0.95 were extended to 1024.

Figure 1 shows each index's best trade-off curve, and Table 2 the operating points at 0.95.

Results. HNSW is the fastest index at 0.95 on both datasets: 155× exact search on SIFT and 15× on GloVe, against 55× and 9× for IVF-Flat. The order is the same at 0.90 (Table 2): 279× and 98× exact on SIFT, 36× and 29× on GloVe.

Why IVF-PQ fails. IVF-PQ never reaches 0.90 (Table 2). It ranks candidates by distances computed on quantised codes, and that quantisation error does not depend on nprobe. When the true 10th and 11th neighbours are closer together than the error, they are mis-ordered whatever nprobe is. Only exact re-ranking can fix this (Part 3).

4 Part 3: Compression on SIFT

Experiment. With nlist = 4096 and 8-bit codes, we vary the number of PQ sub-vectors $m \in \{8, 16, 32, 64\}$. Each m is swept in two variants: plain IVF-PQ, and the *same* trained index wrapped in `IndexRefineFlat`, which re-ranks the top 100 candidates with exact distances. The hypothesis was committed before the run.

Results (Figure 2, Table 3).

- Plain IVF-PQ's recall ceiling rises with m but never reaches 0.90.
- Re-ranking lifts it to 0.95 or above for $m \geq 16$.
- At the same nprobe, re-ranking costs a factor of 1.09 to 1.29 in QPS (median by m), and at most 1.42.
- The price is memory: re-ranking keeps the raw float32 vectors in RAM, about 512 B on top of the PQ part. That erases most of PQ's saving, unless the vectors are

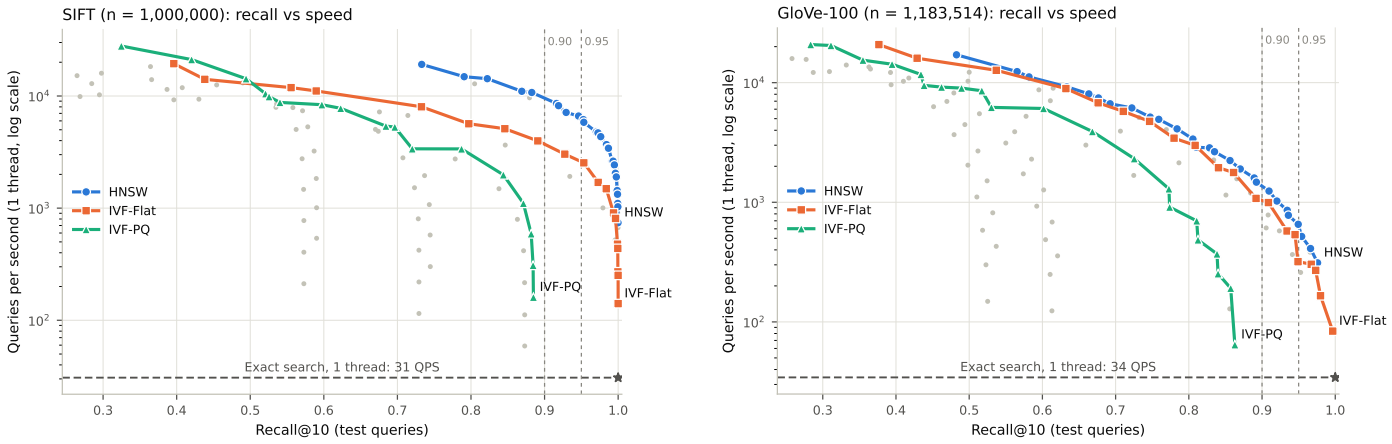


Figure 1: Recall@10 (test) vs single-thread QPS (log): Pareto curve per index over all build settings; dashed line: exact search.

Data	Index	QPS @0.90 (config)	Config @0.95	QPS @0.95	Recall @0.95	Build s	Size MB	B/vector
SIFT	HNSW	8,581 (M16/ef32)	M16/ef64	4,775	0.970	60.8	625.9	656.3
SIFT	IVF-Flat	3,030 (nl4096/np32)	nl4096/np64	1,697	0.973	18.9	497.9	522.1
SIFT	IVF-PQ	not reached	nl4096/m64/np512	not reached (max recall 0.885)	-	62.3	70.8	74.3
GloVe	HNSW	1,241 (M48/ef128)	M32/ef384	518	0.954	149.2	758.6	672.2
GloVe	IVF-Flat	995 (nl8192/np256)	nl4096/np512	303	0.967	18.1	462.1	409.4
GloVe	IVF-PQ	not reached	nl1024/m50/np512	not reached (max recall 0.863)	-	35.4	66.0	58.4

Table 2: Single-thread test QPS (median of 3) at recall@10 0.90 and 0.95, and test recall, build time (20 threads) and index size at the 0.95 config. Configs chosen on tune queries; IVF-PQ shows its highest-recall config.

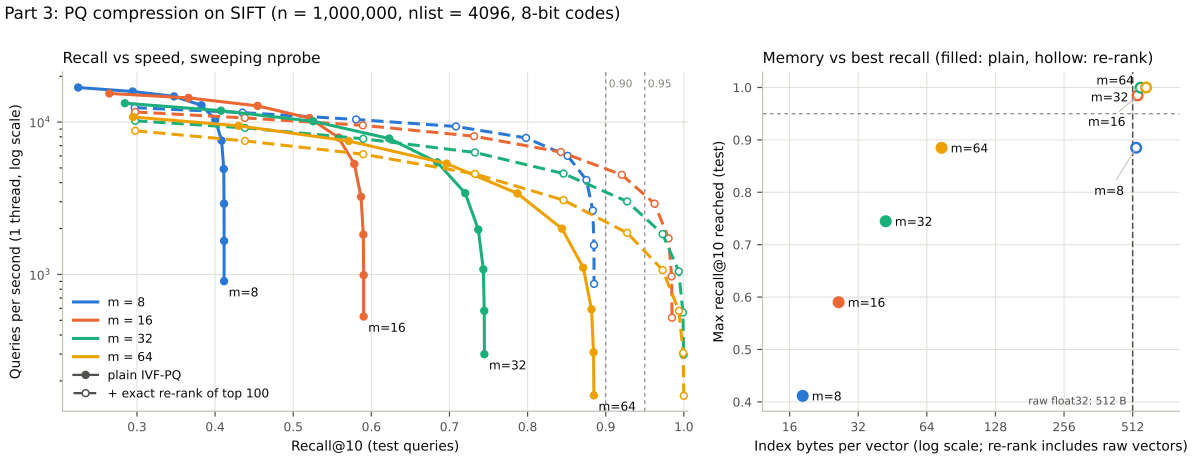


Figure 2: Part 3. Left: recall vs QPS per m (solid: plain; dashed: exact re-rank of the top 100). Right: bytes per vector vs best recall.

m	Plain max R	Re-rank max R	Re-rank QPS @0.95	B/vec RAM	B/vec PQ
8	0.412	0.885	not reached	530.3	18.3
16	0.590	0.985	2,917 (np64)	538.3	26.3
32	0.745	1.000	1,841 (np64)	554.3	42.3
64	0.885	1.000	1,067 (np64)	586.3	74.3

Table 3: SIFT, nlist 4096: best recall of plain IVF-PQ vs re-ranking the top 100; re-rank memory with raw vectors in RAM, and the PQ part alone.

read from disk, which we did not test.

5 Part 4: Drifting data (own data)

What changed.

- Indexes are built on SIFT base vectors 0–200K.
- Vectors 200K–400K are then added, moved by $s \cdot R \cdot u$. Here u is a fixed random unit direction (seed 42) and $R = 383.2$ is the base vectors' mean distance to their centroid.
- 2,000 queries (1,000 tune, 1,000 test) get the same shift. Only s changes: $s \in \{0.25, 0.5, 1\}$, plus the control $s = 0$, for 400K vectors in total.
- *Stale* indexes (trained or built on the base, then extended) are compared with *rebuilt* ones (built on all 400K), for IVF-Flat (nlist 1024) and HNSW (M=32). Ground truth is recomputed for every shift.

What happened (Figure 3, Table 4).

- *Stale IVF-Flat slows down as s grows.* From $s = 0$ to $s = 1$, its list-size CV rises from 0.44 to 0.76 and the longest list from 3.1 to 8.6 times the mean. The share of added vectors in the largest 1% of lists rises from 3% to 9%. At $s = 1$ it needs 18,842 distance computations per query against 12,693 for rebuilt, and reaches only

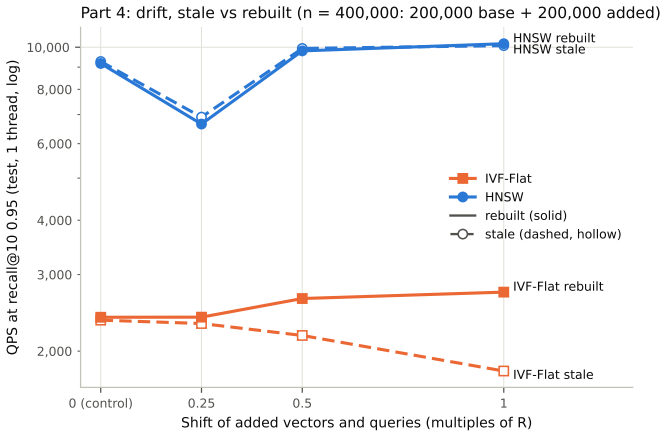


Figure 3: QPS at recall 0.95 vs shift. Solid: rebuilt; dashed, hollow: stale.

Shift	Index	Variant	QPS @0.95	Recall	Dist./q	List CV
0	IVF-Flat	stale	2,357	0.966	14,087	0.44
0	IVF-Flat	rebuilt	2,393	0.968	14,013	0.43
0	HNSW	stale	9,272	0.952	985	-
0	HNSW	rebuilt	9,176	0.951	986	-
1	IVF-Flat	stale	1,800	0.965	18,842	0.76
1	IVF-Flat	rebuilt	2,734	0.979	12,693	0.41
1	HNSW	stale	10,085	0.956	914	-
1	HNSW	rebuilt	10,187	0.954	914	-

Table 4: Drift, shift 0 (control) and 1: test QPS and recall at the 0.95 config, distance computations per query, IVF list-size CV.

0.66 of rebuilt's QPS. Recall stays above 0.95, so the damage is to speed.

- *HNSW is unaffected:* stale and rebuilt are within a QPS ratio of 0.99 to 1.04 at every shift. Inserting the shifted vectors took 7.3 to 10.8 s, against 16.9 to 19.9 s to rebuild.
- *The dip at $s = 0.25$ is a grid step, not drift:* both HNSW variants needed efSearch 48 instead of 32.

Mechanism.

- *IVF:* centroids are frozen at training. Shifted vectors pile into the few cells nearest the new region, and the shifted queries probe exactly those long lists, so each probe scans more vectors. Retraining moves centroids into the new region and rebalances the lists.
- *HNSW:* nothing is trained. Each inserted vector links to its nearest neighbours as it arrives, so the graph adapts.

Hypothesis.

- *Supported:*
  - the control behaves the same stale and rebuilt (QPS ratio 0.98 for IVF, 1.01 for HNSW);
  - IVF imbalance grows with s, and the damage is to speed, not recall;
  - rebuilt IVF stays within 1.00–1.14× of the control;
  - HNSW is robust.
- *Not supported:* the size of the effect. I predicted at least a 2× IVF slowdown at $s = 1$; it was 1.5×.

Not tested: rotations, deletions, new clusters, IVF-PQ, shifts beyond $1.0R$, other nlist and M, and old-distribution queries after drift.

6 Answers

Q1. Fastest at 0.95. HNSW is fastest on both datasets (Table 2), so the winner does not change between datasets, but the cost does. On GloVe both indexes are far slower, and HNSW needs a larger graph and wider search (M=32, efSearch 384 instead of M=16, efSearch 64).

- *Why HNSW wins:* its greedy graph walk moves to ever-closer neighbours, so it computes few distances. IVF scans whole inverted lists, most of which are far from the query. On Part 4's 400K vectors at 0.95, HNSW computed 914–1,337 distances per query against 12,693–18,842 for IVF-Flat.
- *Why GloVe is harder (not measured):* GloVe's nearest neighbours are likely barely closer than many other vectors, so both indexes must examine more candidates.

Q2. Cost; what PQ saves and gives up. Table 2 gives build time and memory.

- *Build time:* HNSW is slowest on GloVe, because every insertion runs a search with efConstruction 200 to find its links. IVF's time is mostly k-means training (13.5 of 18.9 s on SIFT, and 51.7 of IVF-PQ's 62.3 s, which adds PQ codebooks).
- *Memory:* HNSW stores the raw vectors plus about $2M$ neighbour ids, so it needs more than the raw 512 or 400 B per vector. IVF-Flat is the raw vectors plus an 8-byte id.
- *What PQ saves:* it keeps only an m-byte code, so SIFT IVF-PQ takes 18.3 to 74.3 B per vector.
- *What PQ gives up:* accuracy, a ceiling of 0.885 on SIFT and 0.863 on GloVe. Getting recall back by re-ranking puts the raw vectors back in RAM (Table 3).

Q3. Parts 3 and 4. Part 3 against my hypothesis:

- *Supported:*
  - plain IVF-PQ is capped, and the cap rises with m (m=8: 0.412, below my 0.60; m=16: 0.590, below my 0.90);
  - re-ranking reaches 0.95 for $m \geq 16$, while m=8 falls short (0.885), as allowed;
  - re-ranking costs under 2× in QPS;
  - memory was within my estimates (18.3–74.3 vs 16–72 B predicted).
- *Wrong:*
  - m=64 did not "come close to 0.95" (0.885);
  - larger m cost far more than "slightly": re-rank QPS at 0.95 fell from 2,917 (m=16) to 1,067 (m=64).

Part 4 held in direction (control, IVF damage, recovery by rebuilding, robust HNSW) but not in size: the IVF slowdown at $s = 1$ was 1.5×, not 2×.

When headline numbers mislead:

- a PQ "bytes per vector" figure means little without the recall it reaches, because the compact index cannot reach 0.95 and the index that can is no longer compact;
- headline QPS comes from indexes built on the data they serve. Under drift, an extended IVF index is slower than Part 2 suggests, while HNSW is not.

Q4. 150K or 10M vectors, 100 ms, 16 GB (Table 5).

- *150K: no ANN is needed.* Measured exact search takes

Data	n	Exact ms [M/P]	HNSW GiB	Recommendation
SIFT	150,000	4.5 [M]	0.09	Exact
SIFT	10,000,000	324.9 [P]	6.11	HNSW M16/ef64
GloVe	150,000	3.3 [M]	0.09	Exact
GloVe	10,000,000	245.8 [P]	6.26	HNSW M32/ef384

Table 5: Q4: exact single-thread latency (M measured, P projected linearly in n), HNSW memory at the 0.95 config, recommended index (100 ms, 16 GB).

4.5 and 3.3 ms with recall 1.0. The linear model from the 1M timings predicted 4.87 and 3.69 ms, so latency scales about linearly with n. ANN pays off only if throughput matters.

- *10M: use HNSW.* Exact search is *projected* at 324.9 and 245.8 ms, because every query scans all vectors. Memory is not the problem: the raw vectors alone take 4.77 and 3.73 GiB, so exact search fails on latency, not RAM. HNSW at the measured 0.95 settings is *extrapolated* to 0.24 and 2.22 ms, since its search grows roughly with $\log n$. It needs 6.11 and 6.26 GiB, which fits.
- *Caveats at 10M:*
  - building needs the raw vectors and the graph in RAM at once (about 10.9 and 10.0 GiB);
  - build time extrapolates to about 12 and 24 min, optimistically;
  - efSearch must rise to hold 0.95 (not measured);
  - IVF-PQ with re-ranking from disk is the fallback if memory is tight.

Q5. How this benchmark could mislead.

1. *Machine-state drift (tested, not corrected).*
   - The overnight canary varied by 21.3% (SIFT) and 23.3% (GloVe).
   - Power state matters too: the quick pipeline ran much slower on battery than plugged in (see the README).
   - Pinning and medians cut noise within a measurement, not across hours. Part 4's close-in-time control agreed within 2%.
2. *"Batch" speed is a library heuristic (tested).*
   - FAISS 1.14.3 uses BLAS only when queries $\times d \geq$128,000.
   - A single-thread batch over 100K GloVe vectors ran at 216 QPS with 1,279 queries but 1,306 with 1,280 (timed on battery; recall 1.0000 for both).
   - Our 5,000-query batches are above the threshold.
3. *Single thread, one machine (not tested elsewhere).* 20 threads helped SIFT batches (761 vs 342 QPS) far more than GloVe (383 vs 348). Rankings under multi-threaded serving or on other CPUs are untested.
4. *Tune/test split (tested).* The $|\text{test} - \text{tune}|$ recall gap has a median of 0.0147 (SIFT) and 0.0043 (GloVe), at most 0.0500. GloVe HNSW M=16/efSearch 1024 reaches 0.9527 on test but 0.9490 on tune, so it does not count as reaching 0.95.
5. *Tie tolerance (partly tested).* 65 SIFT queries tie at the 10th neighbour, so a tie-blind check scores exact search at 0.9935 instead of 1.0000. Ties are unit-tested; sensitivity to ϵ is not.